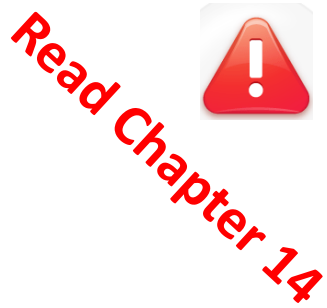


Unit 2-02

String, StringBuilder, and Character classes

Outline

- String class
- StringBuilder class
- Character class

Basic concepts

- The Character, String, and StringBuilder classes are in the **java.lang** package.

- A program may contain character literals.
  - An integer value represented as a character in single quotes.
  - The value of a character literal is the integer value of the character in the Unicode character set.

- **String** literals (stored in memory as **String** objects) are written as a sequence of characters in double quotation marks.

Class String constructors

- No-argument constructor:
  - creates a String that contains no characters (i.e., the empty string, which can also be represented as "") and has a length of 0.

- Constructor that takes a String object:
  - copies the argument into the new String.

- Constructor that takes a char array:
  - creates a String containing a copy of the characters in the array.

- Constructor that takes a char array and two integers:
  - creates a String containing the specified portion of the array.

```java
// Fig. 14.1: StringConstructors.java
// String class constructors.

public class StringConstructors {
   public static void main(String[] args) {
      char[] charArray = {'b', 'i', 'r', 't', 'h', ' ', 'd', 'a', 'y'};
      String s = new String("hello");

      // use String constructors
      String s1 = new String();
      String s2 = new String(s);
      String s3 = new String(charArray);
      String s4 = new String(charArray, 6, 3);

      System.out.printf(
         "s1 = %s%ns2 = %s%ns3 = %s%ns4 = %s%n", s1, s2, s3, s4);
   }
}
```

```
s1 =
s2 = hello
s3 = birth day
s4 = day
```

Fig. 14.1 | `String` class constructors.

Important remarks

- String objects are **immutable**.
  - Their character contents cannot be changed after they created.
  - Class String does not provide methods that allow the contents of a String object to be modified.

- Accessing a character outside the bounds of a String (index less than 0 or greater or equal to the length of the string) results in an exception:

  StringIndexOutOfBoundException

Methods of the class String

- **length**: determines the number of characters in a string.
- **charAt**: returns the character at a specific position in the String.
- **getChars**: copies the characters of a String into a character array.
  - The first argument is the starting index in the String from which characters are to be copied.
  - The second argument is the index that is one past the last character to be copied from the String.
  - The third argument is the character array into which the characters are to be copied.
  - The last argument is the starting index where the copied characters are placed in the target character array.

```
1  // Fig. 14.2: StringMiscellaneous.java
2  // This application demonstrates the length, charAt and getChars
3  // methods of the String class.
4
5  public class StringMiscellaneous {
6     public static void main(String[] args) {
7        String s1 = "hello there";
8        char[] charArray = new char[5];
9
10       System.out.printf("s1: %s", s1);
11
12       // test length method
13       System.out.printf("%nLength of s1: %d", s1.length());
14
15       // loop through characters in s1 with charAt and display reversed
16       System.out.printf("%nThe string reversed is: ");
17
18       for (int count = s1.length() - 1; count >= 0; count--) {
19          System.out.printf("%c ", s1.charAt(count));
20       }
```

Fig. 14.2 | `String` methods `length`, `charAt` and `getChars`. (Part 1 of 2.)

```
21
22          // copy characters from string into charArray
23          s1.getChars(0, 5, charArray, 0);
24          System.out.printf("%nThe character array is: ");
25
26          for (char character : charArray) {
27             System.out.print(character);
28          }
29
30          System.out.println();
31       }
32    }
```

```
s1: hello there
Length of s1: 11
The string reversed is: e r e h t   o l l e h
The character array is: hello
```

Fig. 14.2 | `String` methods `length`, `charAt` and `getChars`. (Part 2 of 2.)

Comparing Strings

- Strings are compared using the numeric codes of the characters in the strings.

- Various comparisons
  - **equals**
  - **equalsIgnoreCase**
  - **compareTo**
  - **regionMatches**
  - using the equality operator **==** to compare String objects

Comparing Strings

- Method **equals**:
  - tests any two objects for equality
  - The method returns true if the contents of the objects are equal, and false otherwise.
  - Uses a lexicographical comparison.

- When primitive-type values are compared with **==**, the result is true if both values are identical.

- When references are compared with **==**, the result is true if both references refer to the same object in memory.

- When comparing objects to determine whether they have the same contents, use method **equals**.

Comparing Strings

- Method **equalsIgnoreCase**:
  - ignores whether the letters in each String are uppercase or lowercase when performing the comparison.
- Method **compareTo**:
  - is declared in the Comparable interface and implemented in the String class.
  - Returns:
    - 0 if the Strings are equal,
    - a negative number if the String that invokes **compareTo** is less than the String that is passed as an argument, and
    - a positive number if the String that invokes **compareTo** is greater than the String that is passed as an argument

Comparing Strings

- Method **regionMatches**:
  - compares portions of two Strings for equality.
  - The first argument is the starting index in the String that invokes the method.
  - The second argument is a comparison String.
  - The third argument is the starting index in the comparison String.
  - The last argument is the number of characters to compare.
- Five-argument version of method **regionMatches**:
  - When the first argument is true, the method ignores the case of the characters being compared.
  - The remaining arguments are identical to those described for the four-argument **regionMatches** method.

```
1  // Fig. 14.3: StringCompare.java
2  // String methods equals, equalsIgnoreCase, compareTo and regionMatches.
3
4  public class StringCompare {
5     public static void main(String[] args) {
6        String s1 = new String("hello"); // s1 is a copy of "hello"
7        String s2 = "goodbye";
8        String s3 = "Happy Birthday";
9        String s4 = "happy birthday";
10
11       System.out.printf(
12          "s1 = %s%ns2 = %s%ns3 = %s%ns4 = %s%n%n", s1, s2, s3, s4);
13
```

Fig. 14.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 1 of 5.)

```
14          // test for equality
15          if (s1.equals("hello")) {  // true
16             System.out.println("s1 equals \"hello\"");
17          }
18          else {
19             System.out.println("s1 does not equal \"hello\"");
20          }
21
22          // test for equality with ==
23          if (s1 == "hello") { // false; they are not the same object
24             System.out.println("s1 is the same object as \"hello\"");
25          }
26          else {
27             System.out.println("s1 is not the same object as \"hello\"");
28          }
29
```

Fig. 14.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 2 of 5.)

```
30          // test for equality (ignore case)
31          if (s3.equalsIgnoreCase(s4)) { // true
32             System.out.printf("%s equals %s with case ignored%n", s3, s4);
33          }
34          else {
35             System.out.println("s3 does not equal s4");
36          }
37
38          // test compareTo
39          System.out.printf(
40             "%ns1.compareTo(s2) is %d", s1.compareTo(s2));
41          System.out.printf(
42             "%ns2.compareTo(s1) is %d", s2.compareTo(s1));
43          System.out.printf(
44             "%ns1.compareTo(s1) is %d", s1.compareTo(s1));
45          System.out.printf(
46             "%ns3.compareTo(s4) is %d", s3.compareTo(s4));
47          System.out.printf(
48             "%ns4.compareTo(s3) is %d%n%n", s4.compareTo(s3));
```

Fig. 14.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 3 of 5.)

```
49
50          // test regionMatches (case sensitive)
51          if (s3.regionMatches(0, s4, 0, 5)) {
52             System.out.println("First 5 characters of s3 and s4 match");
53          }
54          else {
55             System.out.println(
56                "First 5 characters of s3 and s4 do not match");
57          }
58
59          // test regionMatches (ignore case)
60          if (s3.regionMatches(true, 0, s4, 0, 5)) {
61             System.out.println(
62                "First 5 characters of s3 and s4 match with case ignored");
63          }
64          else {
65             System.out.println(
66                "First 5 characters of s3 and s4 do not match");
67          }
68       }
69    }
```

Fig. 14.3 | `String` methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`. (Part 4 of 5.)

Class String methods

- methods **startsWith** and **endsWith**
  - determine whether strings start with or end with a particular set of characters.

- versions of String methods **indexOf** and **lastIndexOf**:
  - search for a specified character or substring in a String.

```
1  // Fig. 14.4: StringStartEnd.java
2  // String methods startsWith and endsWith.
3
4  public class StringStartEnd {
5     public static void main(String[] args) {
6        String[] strings = {"started", "starting", "ended", "ending"};
7
8        // test method startsWith
9        for (String string : strings) {
10          if (string.startsWith("st")) {
11             System.out.printf("\"%s\" starts with \"st\"%n", string);
12          }
13       }
14
15       System.out.println();
16
```

Fig. 14.4 | `String` methods `startsWith` and `endsWith`. (Part 1 of 3.)

```
17          // test method startsWith starting from position 2 of string
18          for (String string : strings) {
19             if (string.startsWith("art", 2)) {
20                System.out.printf(
21                   "\"%s\" starts with \"art\" at position 2%n", string);
22             }
23          }
24
25          System.out.println();
26
27          // test method endsWith
28          for (String string : strings) {
29             if (string.endsWith("ed")) {
30                System.out.printf("\"%s\" ends with \"ed\"%n", string);
31             }
32          }
33       }
34    }
```

Fig. 14.4 | `String` methods `startsWith` and `endsWith`. (Part 2 of 3.)

Class String methods

- Method **indexOf** with one integer argument:
  - locates the first occurrence of a character in a String. If the method finds the character, it returns the character's index in the String—otherwise, it returns –1.
- Method **indexOf** with two integer arguments:
  - the character and the starting index at which the search of the String should begin.
- Method **lastIndexOf** with one integer argument:
  - locates the last occurrence of a character in a String.
  - The method searches from the end of the String toward the beginning.
  - If it finds the character, it returns the character's index in the String—otherwise, it returns –1.
- Method **lastIndexOf** with two integer arguments:
  - the integer representation of the character and the index from which to begin searching backward.
  - There are also versions of these methods that search for substrings in Strings.

```
1  // Fig. 14.5: StringIndexMethods.java
2  // String searching methods indexOf and lastIndexOf.
3
4  public class StringIndexMethods {
5     public static void main(String[] args) {
6        String letters = "abcdefghijklmabcdefghijklm";
7
8        // test indexOf to locate a character in a string
9        System.out.printf(
10          "'c' is located at index %d%n", letters.indexOf('c'));
11       System.out.printf(
12          "'a' is located at index %d%n", letters.indexOf('a', 1));
13       System.out.printf(
14          "'$' is located at index %d%n%n", letters.indexOf('$'));
15
```

Fig. 14.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 1 of 4.)

```
16          // test lastIndexOf to find a character in a string
17          System.out.printf("Last 'c' is located at index %d%n",
18             letters.lastIndexOf('c'));
19          System.out.printf("Last 'a' is located at index %d%n",
20             letters.lastIndexOf('a', 25));
21          System.out.printf("Last '$' is located at index %d%n%n",
22             letters.lastIndexOf('$'));
23
24          // test indexOf to locate a substring in a string
25          System.out.printf("\"def\" is located at index %d%n",
26             letters.indexOf("def"));
27          System.out.printf("\"def\" is located at index %d%n",
28             letters.indexOf("def", 7));
29          System.out.printf("\"hello\" is located at index %d%n%n",
30             letters.indexOf("hello"));
```

Fig. 14.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 2 of 4.)

```
31
32          // test lastIndexOf to find a substring in a string
33          System.out.printf("Last \"def\" is located at index %d%n",
34             letters.lastIndexOf("def"));
35          System.out.printf("Last \"def\" is located at index %d%n",
36             letters.lastIndexOf("def", 25));
37          System.out.printf("Last \"hello\" is located at index %d%n",
38             letters.lastIndexOf("hello"));
39       }
40    }
```

Fig. 14.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 3 of 4.)

Class String methods

- Method **substring**:
  - enables a new String object to be created by copying part of an existing String object.
  - The method returns a new String object.
  - With one integer argument:
    - specifies the starting index in the original String from which characters are to be copied.
  - With two integer arguments:
    - the starting index from which to copy characters in the original String and the index one beyond the last character to copy.

```
1  // Fig. 14.6: SubString.java
2  // String class substring methods.
3
4  public class SubString {
5     public static void main(String[] args) {
6        String letters = "abcdefghijklmabcdefghijklm";
7
8        // test substring methods
9        System.out.printf("Substring from index 20 to end is \"%s\"%n",
10          letters.substring(20));
11       System.out.printf("%s \"%s\"%n",
12          "Substring from index 3 up to, but not including, 6 is",
13          letters.substring(3, 6));
14    }
15 }
```

```
Substring from index 20 to end is "hijklm"
Substring from index 3 up to, but not including, 6 is "def"
```

Fig. 14.6 | `String` class `substring` methods.

Class String methods

- Method **concat**:
  - concatenates two String objects
  - returns a new String object containing the characters from both original Strings.
  - The original Strings are not modified.
- Method **replace**:
  - return a new String object in which every occurrence of the first char argument is replaced with the second.
  - An overloaded version enables you to replace substrings rather than individual characters.

Class String methods

- Method **toUpperCase**:
  - generates a new String with uppercase letters.
- Method **toLowerCase**:
  - returns a new String object with lowercase letters.
- Method **trim**:
  - generates a new String object that removes all whitespace characters that appear at the beginning or end of the String on which trim operates.
- Method **toCharArray**:
  - creates a new character array containing a copy of the characters in the String.

```
1  // Fig. 14.7: StringConcatenation.java
2  // String method concat.
3
4  public class StringConcatenation {
5     public static void main(String[] args) {
6        String s1 = "Happy ";
7        String s2 = "Birthday";
8
9        System.out.printf("s1 = %s%ns2 = %s%n%n",s1, s2);
10       System.out.printf(
11          "Result of s1.concat(s2) = %s%n", s1.concat(s2));
12       System.out.printf("s1 after concatenation = %s%n", s1);
13    }
14 }
```

```
s1 = Happy
s2 = Birthday

Result of s1.concat(s2) = Happy Birthday
s1 after concatenation = Happy
```

Fig. 14.7 | `String` method `concat`.

```
1  // Fig. 14.8: StringMiscellaneous2.java
2  // String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
3
4  public class StringMiscellaneous2 {
5     public static void main(String[] args) {
6        String s1 = "hello";
7        String s2 = "GOODBYE";
8        String s3 = "   spaces   ";
9
10       System.out.printf("s1 = %s%ns2 = %s%ns3 = %s%n%n", s1, s2, s3);
11
12       // test method replace
13       System.out.printf(
14          "Replace 'l' with 'L' in s1: %s%n%n", s1.replace('l', 'L'));
15
16       // test toLowerCase and toUpperCase
17       System.out.printf("s1.toUpperCase() = %s%n", s1.toUpperCase());
18       System.out.printf("s2.toLowerCase() = %s%n%n", s2.toLowerCase());
```

Fig. 14.8 | `String` methods `replace`, `toLowerCase`, `toUpperCase`, `trim` and `toCharArray`. (Part 1 of 3.)

```
19
20          // test trim method
21          System.out.printf("s3 after trim = \"%s\"%n%n", s3.trim());
22
23          // test toCharArray method
24          char[] charArray = s1.toCharArray();
25          System.out.print("s1 as a character array = ");
26
27          for (char character : charArray) {
28             System.out.print(character);
29          }
30
31          System.out.println();
32       }
33    }
```

Fig. 14.8 | `String` methods `replace`, `toLowerCase`, `toUpperCase`, `trim` and `toCharArray`. (Part 2 of 3.)

Class String methods

- Static method **valueOf**:
  - takes an argument of any type and convert it to a String object.

```
1  // Fig. 14.9: StringValueOf.java
2  // String valueOf methods.
3
4  public class StringValueOf {
5     public static void main(String[] args) {
6        char[] charArray = {'a', 'b', 'c', 'd', 'e', 'f'};
7        boolean booleanValue = true;
8        char characterValue = 'Z';
9        int integerValue = 7;
10       long longValue = 10000000000L; // L suffix indicates long
11       float floatValue = 2.5f; // f indicates that 2.5 is a float
12       double doubleValue = 33.333; // no suffix, double is default
13       Object objectRef = "hello"; // assign string to an Object reference
14
15       System.out.printf(
16          "char array = %s%n", String.valueOf(charArray));
17       System.out.printf("part of char array = %s%n",
18          String.valueOf(charArray, 3, 3));
```

Fig. 14.9 | String valueOf methods. (Part 1 of 3.)

```
19          System.out.printf(
20             "boolean = %s%n", String.valueOf(booleanValue));
21          System.out.printf(
22             "char = %s%n", String.valueOf(characterValue));
23          System.out.printf("int = %s%n", String.valueOf(integerValue));
24          System.out.printf("long = %s%n", String.valueOf(longValue));
25          System.out.printf("float = %s%n", String.valueOf(floatValue));
26          System.out.printf(
27             "double = %s%n", String.valueOf(doubleValue));
28          System.out.printf("Object = %s", String.valueOf(objectRef));
29       }
30    }
```

Fig. 14.9 | `String` `valueOf` methods. (Part 2 of 3.)

Class Character Methods

- Method **isDefined**:
  - determines whether a character is defined in the Unicode character set. (Skip-most are Char)
- Method **isDigit**:
  - determines whether a character is a defined Unicode digit.
- Method **isJavaIdentifierStart**:
  - determines whether a character can be the first character of an identifier in Java—that is, a letter, an underscore (_) or a dollar sign ($).
- Method **isJavaIdentifierPart**:
  - determine whether a character can be used in an identifier in Java—that is, a digit, a letter, an underscore (_) or a dollar sign ($).

Class Character Methods

- Method **isLetter**:
  - determines whether a character is a letter.
- Method **isLetterOrDigit**:
  - determines whether a character is a letter or a digit.
- Method **isLowerCase**:
  - determines whether a character is a lowercase letter.
- Method **isUpperCase**:
  - determines whether a character is an uppercase letter.
- Method **toUpperCase**:
  - converts a character to its uppercase equivalent.
- Method **toLowerCase**:
  - converts a character to its lowercase equivalent.

```
1  // Fig. 14.15: StaticCharMethods.java
2  // Character static methods for testing characters and converting case.
3  import java.util.Scanner;
4
5  public class StaticCharMethods {
6     public static void main(String[] args) {
7        Scanner scanner = new Scanner(System.in); // create scanner
8        System.out.println("Enter a character and press Enter");
9        String input = scanner.next();
10       char c = input.charAt(0); // get input character
11
12       // display character info
13       System.out.printf("is defined: %b%n", Character.isDefined(c));
14       System.out.printf("is digit: %b%n", Character.isDigit(c));
15       System.out.printf("is first character in a Java identifier: %b%n",
16          Character.isJavaIdentifierStart(c));
```

Fig. 14.15 | `Character` `static` methods for testing characters and converting case. (Part 1 of 5.)

```
17          System.out.printf("is part of a Java identifier: %b%n",
18             Character.isJavaIdentifierPart(c));
19          System.out.printf("is letter: %b%n", Character.isLetter(c));
20          System.out.printf(
21             "is letter or digit: %b%n", Character.isLetterOrDigit(c));
22          System.out.printf(
23             "is lower case: %b%n", Character.isLowerCase(c));
24          System.out.printf(
25             "is upper case: %b%n", Character.isUpperCase(c));
26          System.out.printf(
27             "to upper case: %s%n", Character.toUpperCase(c));
28          System.out.printf(
29             "to lower case: %s%n", Character.toLowerCase(c));
30       }
31    }
```

Fig. 14.15 | `Character` `static` methods for testing characters and converting case. (Part 2 of 5.)

Class Character Methods

- Methods **digit** and **forDigit**:
  - convert characters to digits and digits to characters, respectively, in different number systems.
- Common number systems:
  - decimal (base 10), octal (base 8), hexadecimal (base 16) and binary (base 2).
  - The base of a number is also known as its **radix**.
  - For more information on conversions between number systems, **see Appendix H**.

Class Character Methods

- Method **forDigit**:
  - converts its first argument into a character in the number system specified by its second argument.
- Method **digit**:
  - converts its first argument into an integer in the number system specified by its second argument.
  - The radix (second argument) must be between 2 and 36, inclusive.

```
1  // Fig. 14.16: StaticCharMethods2.java
2  // Character class static conversion methods.
3  import java.util.Scanner;
4
5  public class StaticCharMethods2 {
6     public static void main(String[] args) {
7        Scanner scanner = new Scanner(System.in);
8
9        // get radix
10       System.out.println("Please enter a radix:");
11       int radix = scanner.nextInt();
12
13       // get user choice
14       System.out.printf("Please choose one:%n1 -- %s%n2 -- %s%n",
15          "Convert digit to character", "Convert character to digit");
16       int choice = scanner.nextInt();
17
```

Fig. 14.16 | `Character` class `static` conversion methods. (Part 1 of 3.)

```
18          // process request
19          switch (choice) {
20             case 1: // convert digit to character
21                System.out.println("Enter a digit:");
22                int digit = scanner.nextInt();
23                System.out.printf("Convert digit to character: %s%n",
24                   Character.forDigit(digit, radix));
25                break;
26             case 2: // convert character to digit
27                System.out.println("Enter a character:");
28                char character = scanner.next().charAt(0);
29                System.out.printf("Convert character to digit: %s%n",
30                   Character.digit(character, radix));
31                break;
32          }
33       }
34    }
```

Fig. 14.16 | `Character` class `static` conversion methods. (Part 2 of 3.)

```
Please enter a radix:
16
Please choose one:
1 -- Convert digit to character
2 -- Convert character to digit
2
Enter a character:
A
Convert character to digit: 10
```

```
Please enter a radix:
16
Please choose one:
1 -- Convert digit to character
2 -- Convert character to digit
1
Enter a digit:
13
Convert digit to character: d
```

Fig. 14.16 | `Character` class `static` conversion methods. (Part 3 of 3.)

Class Character Methods

- Java automatically converts char literals into Character objects when they are assigned to Character variables.
  - Process known as **autoboxing**.
- Method **charValue**:
  - returns the char value stored in the object.
- Method **toString**:
  - returns the String representation of the char value stored in the object.
- Method **equals**:
  - determines if two Characters have the same contents.

```
1  // Fig. 14.17: OtherCharMethods.java
2  // Character class instance methods.
3  public class OtherCharMethods {
4     public static void main(String[] args) {
5        Character c1 = 'A';
6        Character c2 = 'a';
7
8        System.out.printf(
9           "c1 = %s%nc2 = %s%n%n", c1.charValue(), c2.toString());
10
11       if (c1.equals(c2)) {
12          System.out.println("c1 and c2 are equal%n");
13       }
14       else {
15          System.out.println("c1 and c2 are not equal%n");
16       }
17    }
18 }
```

Fig. 14.17 | `Character` class instance methods. (Part 1 of 2.)

Tokenizing Strings

- When you read a sentence, your mind breaks it into **tokens**
  - individual words and punctuation marks that convey meaning.
- Compilers also perform **tokenization**.
- String method **split**:
  - breaks a String into its component tokens
  - returns an array of Strings.
- Tokens are separated by **delimiters**
  - Typically white-space characters such as space, tab, newline and carriage return.
  - Other characters can also be used as delimiters to separate tokens.

```
1  // Fig. 14.18: TokenTest.java
2  // Tokenizing with String method split
3  import java.util.Scanner;
4
5  public class TokenTest {
6     // execute application
7     public static void main(String[] args) {
8        // get sentence
9        Scanner scanner = new Scanner(System.in);
10       System.out.println("Enter a sentence and press Enter");
11       String sentence = scanner.nextLine();
12
13       // process user sentence
14       String[] tokens = sentence.split(" ");
15       System.out.printf("Number of elements: %d%nThe tokens are:%n",
16          tokens.length);
17
18       for (String token : tokens) {
19          System.out.println(token);
20       }
21    }
22 }
```

Fig. 14.18 | Tokenizing with `String` method `split`. (Part 1 of 2.)

StringBuilder class

- Used to create and manipulate dynamic string information.
  - Modifiable strings
  - Use it when you have lots of string concatenation or modification.
- Every **StringBuilder** is capable of storing a number of characters specified by its capacity.
- If the capacity of a **StringBuilder** is exceeded, the capacity expands to accommodate the additional characters.

StringBuilder constructors

- No-argument constructor:
  - creates a **StringBuilder** with no characters; initial capacity is 16.
- Constructor that takes an integer argument :
  - creates a **StringBuilder** with no characters in it and the initial capacity specified by the integer argument.
- Constructor that takes a String argument:
  - creates a **StringBuilder** containing the characters in the String argument. The initial capacity is the number of characters in the String argument plus 16.
- Method **toString** of class **StringBuilder** returns the **StringBuilder** contents as a **String**.

```
1  // Fig. 14.10: StringBuilderConstructors.java
2  // StringBuilder constructors.
3
4  public class StringBuilderConstructors {
5     public static void main(String[] args) {
6        StringBuilder buffer1 = new StringBuilder();
7        StringBuilder buffer2 = new StringBuilder(10);
8        StringBuilder buffer3 = new StringBuilder("hello");
9
10       System.out.printf("buffer1 = \"%s\"%n", buffer1);
11       System.out.printf("buffer2 = \"%s\"%n", buffer2);
12       System.out.printf("buffer3 = \"%s\"%n", buffer3);
13    }
14 }
```

```
buffer1 = ""
buffer2 = ""
buffer3 = "hello"
```

Fig. 14.10 | `StringBuilder` constructors.

StringBuilder Methods

- Methods **length** and **capacity** :
  - return the number of characters currently in a **StringBuilder** and the number of characters that can be stored in a without allocating more memory, respectively.
- Method **ensureCapacity**:
  - guarantees that a StringBuilder has at least the specified capacity.
- Method **setLength**:
  - increases or decreases the length of a **StringBuilder**.
  - If the specified length is less than the current number of characters, the buffer is truncated to the specified length.
  - If the specified length is greater than the number of characters, null characters are appended until the total number of characters in the **StringBuilder** is equal to the specified length.

```
1  // Fig. 14.11: StringBuilderCapLen.java
2  // StringBuilder length, setLength, capacity and ensureCapacity methods.
3
4  public class StringBuilderCapLen {
5     public static void main(String[] args) {
6        StringBuilder buffer = new StringBuilder("Hello, how are you?");
7
8        System.out.printf("buffer = %s%nlength = %d%ncapacity = %d%n%n",
9           buffer.toString(), buffer.length(), buffer.capacity());
10
11       buffer.ensureCapacity(75);
12       System.out.printf("New capacity = %d%n%n", buffer.capacity());
13
14       buffer.setLength(10));
15       System.out.printf("New length = %d%nbuffer = %s%n",
16          buffer.length(), buffer.toString());
17    }
18 }
```

Fig. 14.11 | `StringBuilder` `length`, `setLength`, `capacity` and `ensureCapacity` methods. (Part 1 of 2.)

StringBuilder Methods

- Method **charAt**:
  - takes an integer argument and returns the character in the StringBuilder at that index.
- Method **getChars**:
  - copies characters from a **StringBuilder** into the character array argument.
  - Four arguments:
    - the starting index from which characters should be copied,
    - the index one past the last character to be copied,
    - the character array into which the characters are to be copied, and
    - the starting location in the character array where the first character should be placed.

StringBuilder Methods

- Method **reverse**:
  - reverses the contents of the **StringBuilder**.
- Method **setCharAt**:
  - takes an integer and a character argument
  - sets the character at the specified position in the **StringBuilder** to the character argument.

```
1  // Fig. 14.12: StringBuilderChars.java
2  // StringBuilder methods charAt, setCharAt, getChars and reverse.
3
4  public class StringBuilderChars {
5     public static void main(String[] args) {
6        StringBuilder buffer = new StringBuilder("hello there");
7
8        System.out.printf("buffer = %s%n", buffer.toString());
9        System.out.printf("Character at 0: %s%nCharacter at 4: %s%n%n",
10          buffer.charAt(0), buffer.charAt(4));
11
12       char[] charArray = new char[buffer.length()];
13       buffer.getChars(0, buffer.length(), charArray, 0);
14       System.out.print("The characters are: ");
15
```

Fig. 14.12 | `StringBuilder` methods `charAt`, `setCharAt`, `getChars` and `reverse`. (Part 1 of 3.)

```
16          for (char character : charArray) {
17             System.out.print(character);
18          }
19
20          buffer.setCharAt(0, 'H');
21          buffer.setCharAt(6, 'T');
22          System.out.printf("%n%nbuffer = %s", buffer.toString());
23
24          buffer.reverse();
25          System.out.printf("%n%nbuffer = %s%n", buffer.toString());
26       }
27    }
```

Fig. 14.12 | `StringBuilder` methods `charAt`, `setCharAt`, `getChars` and `reverse`. (Part 2 of 3.)

StringBuilder Methods

- Overloaded **append** methods:
  - allow values of various types to be appended to the end of a **StringBuilder**.
  - Versions are provided for each of the primitive types, and for character arrays, Strings, Objects, and more.
- A compiler can use **StringBuilder** and the append methods to implement the + and += String concatenation operators.

```
1  // Fig. 14.13: StringBuilderAppend.java
2  // StringBuilder append methods.
3
4  public class StringBuilderAppend
5  {
6     public static void main(String[] args)
7     {
8        Object objectRef = "hello";
9        String string = "goodbye";
10       char[] charArray = {'a', 'b', 'c', 'd', 'e', 'f'};
11       boolean booleanValue = true;
12       char characterValue = 'Z';
13       int integerValue = 7;
14       long longValue = 10000000000L;
15       float floatValue = 2.5f;
16       double doubleValue = 33.333;
17
18       StringBuilder lastBuffer = new StringBuilder("last buffer");
19       StringBuilder buffer = new StringBuilder();
20
21       buffer.append(objectRef)
22             .append(System.getProperty("line.separator"))
23             .append(string)
```

Fig. 14.13 | `StringBuilder` append methods. (Part 1 of 3.)

```
24              .append(System.getProperty("line.separator"))
25              .append(charArray)
26              .append(System.getProperty("line.separator"))
27              .append(charArray, 0, 3)
28              .append(System.getProperty("line.separator"))
29              .append(booleanValue)
30              .append(System.getProperty("line.separator"))
31              .append(characterValue);
32              .append(System.getProperty("line.separator"))
33              .append(integerValue)
34              .append(System.getProperty("line.separator"))
35              .append(longValue)
36              .append(System.getProperty("line.separator"))
37              .append(floatValue)
38              .append(System.getProperty("line.separator"))
39              .append(doubleValue)
40              .append(System.getProperty("line.separator"))
41              .append(lastBuffer);
42
43          System.out.printf("buffer contains%n%s%n", buffer.toString());
44       }
45    }
```

Fig. 14.13 | `StringBuilder` `append` methods. (Part 2 of 3.)

StringBuilder Methods

- Overloaded **insert** methods:
  - insert values of various types at any position in a **StringBuilder**.
  - Versions are provided for the primitive types and for character arrays, Strings, Objects and CharSequences.
  - Each method takes its second argument, converts it to a String and inserts it at the index specified by the first argument.
- Methods **delete** and **deleteCharAt**:
  - delete characters at any position in a **StringBuilder**.
  - Method **delete** takes two arguments:
    - the starting index and
    - the index one past the end of the characters to delete.
  - Method **deleteCharAt** takes one argument:
    - the index of the character to delete

```
1  // Fig. 14.14: StringBuilderInsertDelete.java
2  // StringBuilder methods insert, delete and deleteCharAt.
3
4  public class StringBuilderInsertDelete {
5     public static void main(String[] args) {
6        Object objectRef = "hello";
7        String string = "goodbye";
8        char[] charArray = {'a', 'b', 'c', 'd', 'e', 'f'};
9        boolean booleanValue = true;
10       char characterValue = 'K';
11       int integerValue = 7;
12       long longValue = 10000000;
13       float floatValue = 2.5f; // f suffix indicates that 2.5 is a float
14       double doubleValue = 33.333;
15
16       StringBuilder buffer = new StringBuilder();
17
18       buffer.insert(0, objectRef);
19       buffer.insert(0, "  "); // each of these contains two spaces
20       buffer.insert(0, string);
21       buffer.insert(0, "  ");
22       buffer.insert(0, charArray);
23       buffer.insert(0, "  ");
```

Fig. 14.14 | `StringBuilder` methods `insert`, `delete` and `deleteCharAt`. (Part 1 of 3.)

```
24          buffer.insert(0, charArray, 3, 3);
25          buffer.insert(0, "  ");
26          buffer.insert(0, booleanValue);
27          buffer.insert(0, "  ");
28          buffer.insert(0, characterValue);
29          buffer.insert(0, "  ");
30          buffer.insert(0, integerValue);
31          buffer.insert(0, "  ");
32          buffer.insert(0, longValue);
33          buffer.insert(0, "  ");
34          buffer.insert(0, floatValue);
35          buffer.insert(0, "  ");
36          buffer.insert(0, doubleValue);
37
38          System.out.printf(
39             "buffer after inserts:%n%s%n%n", buffer.toString());
40
41          buffer.deleteCharAt(10); // delete 5 in 2.5
42          buffer.delete(2, 6); // delete .333 in 33.333
43
44          System.out.printf(
45             "buffer after deletes:%n%s%n", buffer.toString());
46       }
47    }
```

Fig. 14.14 | `StringBuilder` methods `insert`, `delete` and `deleteCharAt`. (Part 2 of 3.)